

## Unidad 4

---



# Programacion en STEP 7 (I)

## Sistemas secuenciales programables





# Índice

- 4.1. Introducción
- 4.2. Tamaño de los tipos de datos en STEP 7
- 4.3. Direccionamiento de entradas y salidas
- 4.4. Programación estructurada
- 4.5. Ciclo de SCAN
- 4.6. Variables en STEP 7
- 4.7. Programación en lenguaje de contactos
  - 4.7.1. Símbolos básicos
  - 4.7.2. Asociación de contactos
  - 4.7.3. Funciones lógicas con contactos
  - 4.7.4. Circuitos realimentados
  - 4.7.5. Marcas internas (bytes memories)
  - 4.7.6. Biestables (flip flop)
  - 4.7.7. Detectores de flancos



# Introducción

Hay diferentes tipos de PLC en el mercado, en esta unidad nosotros trabajaremos con el STEP 7 de la marca Siemens. Este PLC tendrá 4 modelos que explicaremos más adelante todo lo que tienen que ver con sus características y similitudes. Programaremos algunas líneas de código básicas con los distintos tipos de variables y bloques que estudiaremos.

## Al finalizar esta unidad

- + Sabremos más acerca del STEP 7 y los lenguajes que tiene
- + Identificaremos las diferentes variables que nos proporciona el STEP 7
- + Aplicaremos conceptos de lógica digital y circuitos secuenciales
- + Realizaremos algún programa con el STEP 7



# 4.1.

## Introducción

Este lenguaje, también conocido como S7, se trata de un conjunto de herramientas que usaremos para programar los autómatas y dispositivos de Siemens. Solamente trabajará con la gama de productos SIMATIC.

Hay diferentes gamas dentro de los PLC de la marca Siemens: S7-300, S7-400, S7-1200, S7-1500. Son prácticamente todos iguales, sobre todo cuando se programa en KOP, excepto por algunas diferencias.

La programación en STEP 7 podremos realizarla tanto en SIMATIC como en Tia Portal. SIMATIC es menos usado ya que es un entorno antiguo que solamente podrá programar modelos S7-300 Y S7-400, mientras que Tia Portal será capaz de programar todos los modelos que tiene la gama SIEMENS.

En la siguiente tabla podremos ver los diferentes modelos de PLC de Siemens y los lenguajes de programación aceptados:

| Gama de PLC                                                                                             | KOP (De contactos) | FUP (Bloques de funcione) | AWL (Lista de instrucciones) | SCL (Texto estructurado) | S7-GRAPH (GRAFCET) |
|---------------------------------------------------------------------------------------------------------|--------------------|---------------------------|------------------------------|--------------------------|--------------------|
| S7-300<br>S7-400<br> | Si                 | Si                        | Si                           | Si                       | Si                 |
| S7-1200<br>          | Si                 | Si                        | No                           | Si                       | No                 |
| S7-1500<br>          | Si                 | Si                        | Si                           | Si                       | Si                 |

# 4.2.

## Tamaño de los tipos de datos en STEP 7

Tiene como función la organización binaria de todos los datos de STEP que tendrán los siguientes formatos:

- > **Bit:** también podremos llamarle bool o booleano, y se trata de la unidad más básica de la información binaria. Tendrá dos estados (0 o 1).
- > **Byte:** Está compuesto de 8 bits.
- > **Word:** Estará compuesto por 2 bytes (16 bits).
- > **Double Word:** consta de dos words (32 bits).



# 4.3.

## Direccionamiento de entradas y salidas

Diremos que el direccionamiento es como una forma de "apuntar" a las distintas posiciones de memoria del PLC.

Este direccionamiento dependerá tanto del modelo como del fabricante del PLC. Se podrá hacer de dos formas: el direccionamiento absoluto (en azul) apuntará a la dirección física del canal y el direccionamiento simbólico (en verde) se trata de una variable que le ponemos el nombre que nosotros deseamos y se le asocia a su valor.



Imagen 1. Direccionamiento de E/S en STEP 7

Podremos acceder a las entradas y salidas con un formato bit, byte o Word. En los Siemens las entradas y salidas irán en grupos de 8 bits (0 al 7).

Podremos definir el direccionamiento absoluto de la siguiente manera:

- > **%**: se indicará cuando vayamos a usar el operando absoluto
- > **Letra**: indicará si es una entrada(I) o salida(Q)
- > **Byte**: indicará el número del grupo al que pertenece el operando
- > **Bit**: número que hará el operando dentro del byte
- > **Punto**: se usará para separar el byte del bit.



## 4.4.

### Programación estructurada

Organizarán los programas en bloques (subrutinas, funciones, ...), que serán llamados y ejecutados cuando se cumplan las condiciones especificadas por el usuario. En STEP 7 los tenemos clasificados en diferentes tipos de bloques:

- > **OB1:** se trata del bloque principal (*main*). Lo ejecutaremos de manera cíclica y podremos llamar a otros bloques desde el mismo. Todos los programas deben tener uno.
- > **FC:** se pueden también llamar *funciones*, tienen programas y pueden llamarlos o llamar a otros bloques.
- > **FB:** se denominan *bloques de función*. Son iguales que los FC, pero deberán tener siempre un bloque de datos (DB) asociado.
- > **DB:** No tienen funciones de programación. Tendrán datos en formato de tabla los cuales podrán leerse o escribirse desde otros bloques del programa. Serán de dos tipos: globales que pueden ser ejecutados desde cualquier parte del programa y los de instancia que estarán asociados a un bloque FB.
- > **OB:** serán bloques especiales que tienen una función de interrupción, de alarma o tratamiento de errores que se ejecutarán cuando sea necesaria su actuación. No hay que llamarlos desde otros bloques, solo se crean y se almacenan en la memoria del PLC. Estos tendrán asignado un número que indique cuál es su funcionamiento.

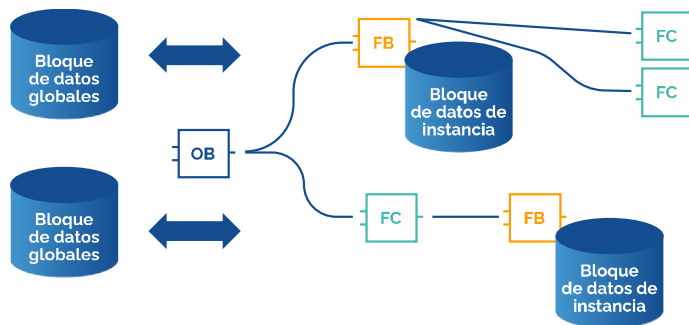


Imagen 2. Programación estructurada en STEP 7

## 4.5.

### Ciclo de SCAN

Cuando hacemos un programa, este tendrá una secuencia de operaciones que se ejecutará de forma repetitiva en su memoria. Normalmente las CPU tienen un tiempo predefinido por defecto que en algunas podría ser ajustado por el usuario.

La ejecución secuencial hará que el ciclo se ejecute de arriba hacia abajo, así que si hay alguna contradicción entre líneas siempre tendrá la preferencia el que más abajo este.



## 4.6.

### Variables en STEP 7

Las variables podrán ser:

- > **Absolutas:** Como ya hemos visto anteriormente, serán aquellas que apuntan a una zona física de la memoria. Por ejemplo: %Q4.3
- > **Simbólicas:** será el nombre simbólico que le dé el usuario y podrán ser globales, las cuales repercutirán en todos los bloques del programa, por ejemplo "Mi\_entrada", o locales que solo repercutirán a bloques, por ejemplo #Mi\_entrada\_local.

Se recomienda al programador para que le resulte más claro y le facilite la programación que haga una tabla con variables que incluya los nombres de estas. Esta tabla se podrá editar e ir incluyendo variables cuando nosotros queramos.

| Variables PLC |               |           |                            |
|---------------|---------------|-----------|----------------------------|
| Nombre        | Tipo de datos | Dirección | Comentario                 |
| Marcha1       | Bool          | %I0.0     | Pulsador de marcha motor 1 |
| Marcha2       | Bool          | %I0.2     | Pulsador de marcha motor 2 |
| Paro          | Bool          | %I0.3     | Paro general               |
| Motor1        | Bool          | %Q0.0     | Motor 1                    |
| Motor2        | Bool          | %Q0.1     | Motor 2                    |

## 4.7.

### Programación en lenguaje de contactos

#### 4.7.1. Símbolos básicos

Los símbolos básicos en este lenguaje de programación serán los siguientes:

- > **Contactos:** tendrán un comportamiento muy parecido a los circuitos eléctricos cableados y podrán estar abiertos o cerrados.
- > **Asignaciones:** muy parecidas a las bobinas de los circuitos eléctricos cableados. Recibirán el valor de una combinación lógica y podrán ser directas o negadas.



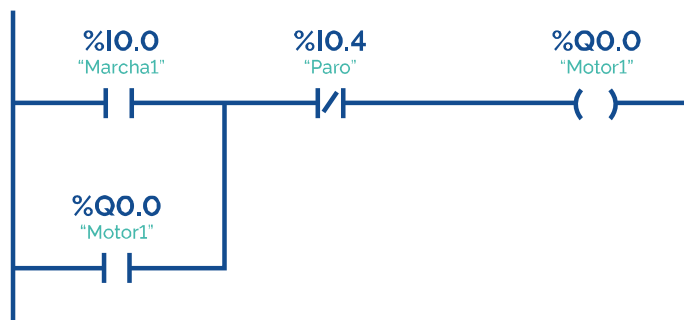


### 4.7.2. Asociación de contactos

Usando los símbolos que hemos visto anteriormente, la programación en lenguaje KOP se hará asociando contactos entre sí y asignando su resultado lógico a las bobinas. Estos contactos podrán asociarse en serie, paralelo o en circuitos mixtos.

Cuando realizamos un programa en KOP tendremos que organizarlo por segmentos. Cada segmento dispondrá de una o más redes de contactos. Debemos tener en cuenta que los modelos S7-1200 y S7-1500 podemos poner redes de contacto interconectadas entre sí, mientras que en el S7-300 y S7-400 no es posible.

#### Segmento 1:



#### Segmento 2:



Imagen 3. Forma de separar redes para S7-300 y S7-400

#### Segmento 1:

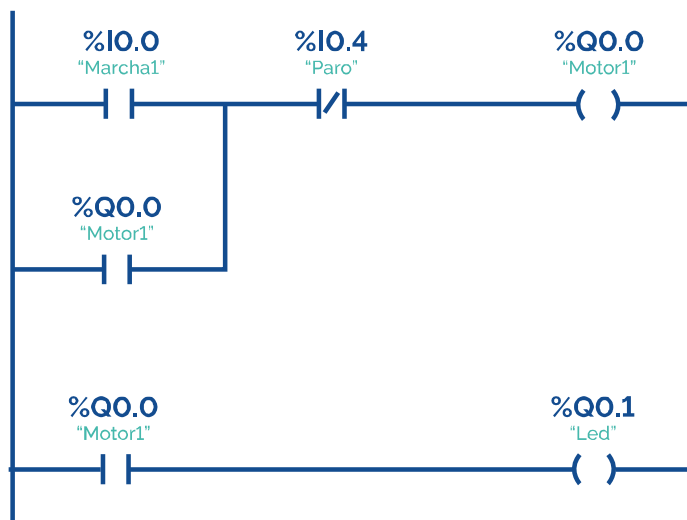


Imagen 4. Forma de separar redes para S7-1200 y S7-1500





### 4.7.3. Funciones lógicas con contactos

#### Conexión serie (AND)

Tendrá su equivalente con el circuito eléctrico combinacional de contactos en serie. Cuando el resultado de la operación de las dos entradas este a "1" simultáneamente se encenderá la bombilla. La ecuación lógica será  $Q=A*B$

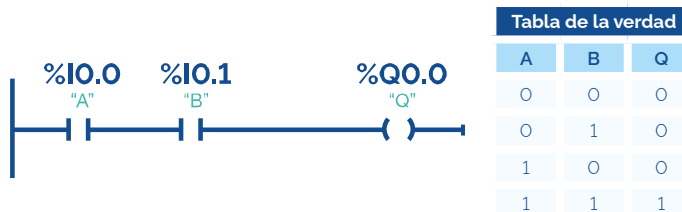


Imagen 5. Asociación AND

#### Conexión paralela (OR)

Tendrá su equivalente con el circuito eléctrico combinacional de contactos en paralelo. Cuando el resultado de la operación de algunas de estas dos entradas sea "1" la bobina se activará. La ecuación lógica es  $Q=A+B$ .

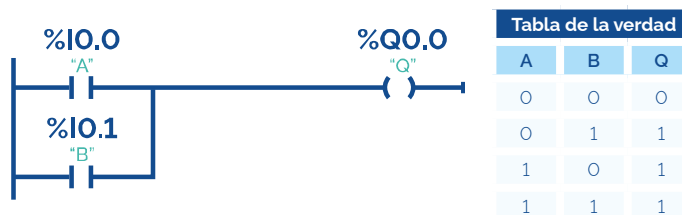


Imagen 6. Asociación OR

#### Operación negación (NOT)

La operación NOT lo que hará será invertir el valor lógico de la operación. En este caso cuando la variable de contacto sea "0" el resultado será "1". Hay varias formas de llamar al NOT en estos PLC, pero nosotros emplearemos la más típica que se usa. Su ecuación lógica será  $Q = \bar{A}$

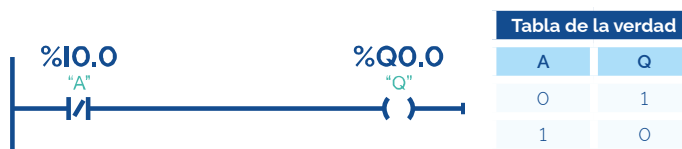


Imagen 7. Asociación NOT



### Operación serie negada (NAND)

Será el resultado de la operación inversa de AND. Se conseguirá con el resultado lógico de contactos en serie. La ecuación lógica será  $Q = \overline{A * B}$

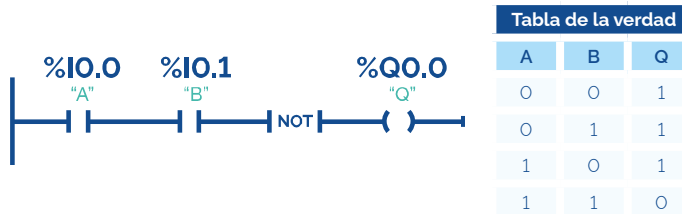


Imagen 8. Asociación NAND

### Operación paralelo negada (NOR)

Es el resultado de la operación inversa NOR. Se conseguirá negando el resultado lógico de los contactos en paralelo. La ecuación lógica será  $Q = \overline{A + B}$

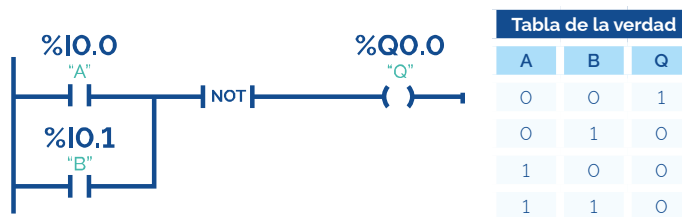


Imagen 9. Asociación NOR

### Agrupaciones de contactos

En un lenguaje KOP podremos representar de forma gráfica tanto agrupaciones de AND como de OR. A continuación, veremos dos ejemplos.

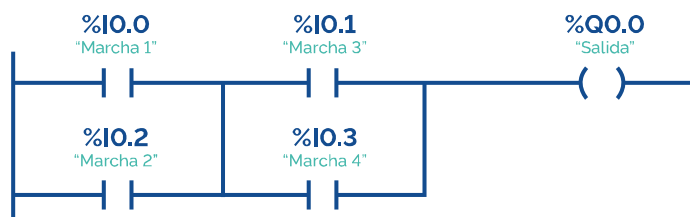


Imagen 10. Dos grupos de contactos en OR conectados entre sí en AND

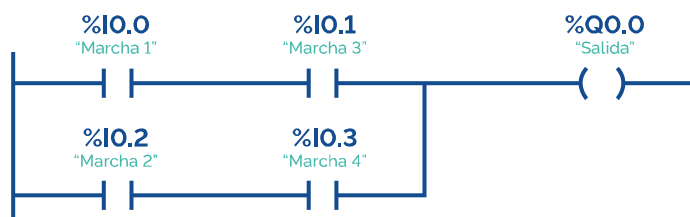


Imagen 11. Dos grupos de contactos en AND conectados entre sí en OR



#### 4.7.4. Circuitos realimentados

Se aplicará de una forma muy parecida a los bloques de funciones lógicas. En este caso, este circuito de realimentación lo haremos poniendo un contacto de salida en paralelo con el contacto.

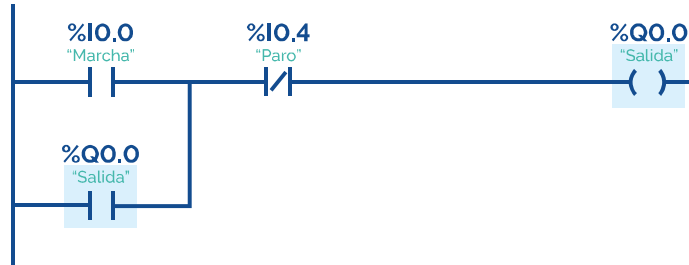


Imagen 12. Ejemplo de programa con realimentación en KOP

#### Activación y desactivación de bobinas

- > **Activación:** los contactos normalmente abiertos de activación se pondrán en paralelo al contacto de la realimentación.
- > **Desactivación:** los contactos normalmente cerrados de activación se pondrán en serie al contacto de la realimentación.

#### 4.7.5. Marcas internas (bytes memories)

En STEP 7, las marcas internas tendrán el mismo patrón que el comentado anteriormente <<byte, bit>> utilizado para entradas y salidas. El identificador se pondrá con la letra M. La cantidad de bytes dependerá del autómatas y tendremos que consultarlo en su hoja de características.

#### Remanencia

La remanencia es un concepto que estará presente en los autómatas programables. Consistirá en configurar alguna de las zonas de la memoria de un PLC (marcas, temporizadores, etc.) para que se mantenga ante un arranque en caliente del dispositivo.

Será necesario para ello que configuremos las marcas internas. Cuando ya hayamos configurado dicho dispositivo, estas marcas mantendrán el estado en el que estaban antes de la parada.



### 4.7.6. Biestables (flip flop)

Como en otros lenguajes de programación, el uso de los biestables en KOP, lo que harán serán activar un bit con una instrucción SET o desactivarla con un RESET.

#### Biestable de bloque

Este bloque se representará en forma de rectángulo y tiene dos entradas (SET y RESET) y una única salida. Los bloques se dividirán en SR y RS. Se deberá usar uno u otro en función de cuál es la preferencia de las entradas si ambas estuviesen a "1" al mismo tiempo. La entrada que más preferencia tendrá será la de abajo.



Imagen 13. Bloque SR

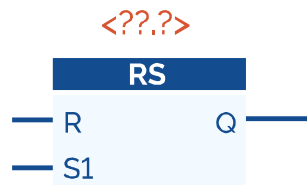


Imagen 14. Bloque RS

#### Bobinas de activación (S) y desactivación (R)

Tendrá el mismo funcionamiento que otro biestable cualquiera, la única diferencia es que la función SET y RESET están separadas como bobinas de asignación individuales. Aporta mucha flexibilidad cuando queremos programar el control de una variable.

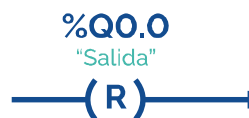


Imagen 15. Bobina de activación y desactivación

#### Activar y desactivar un mapa de bits

Son instrucciones que permiten activar o desactivar varios bits a partir de una dirección. Tienen dos operandos. El de arriba indicará la dirección del bit y en el de abajo la cantidad de bits que habrá que activar o desactivar a partir de él.



Imagen 16. Bloque para activar mapa de bits



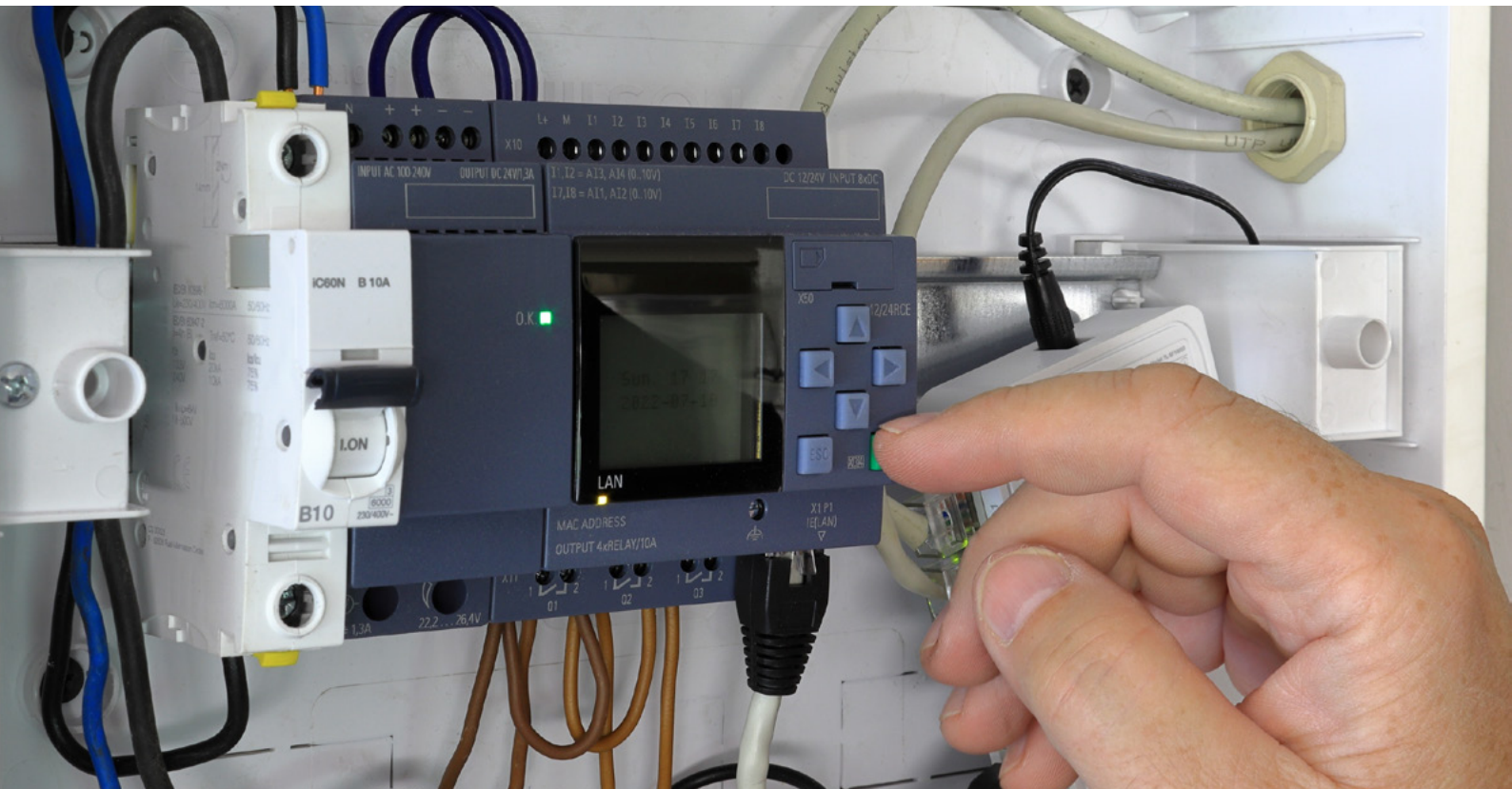
#### 4.7.7. Detectores de flancos

Estos flancos permitirán detectar cuando una función lógica cambia de variable. Si esto ocurre se crea un impulso de una breve duración que podrá ser usada en el programa del usuario para que no tenga señales permanentes.

Los flancos serán positivos (P) o negativos (N). En el lenguaje KOP de STEP 7, los flancos se representarán por contactos especiales con dos operandos. En la parte superior se corresponderá con una señal que quiere detectar el flanco como una entrada, mientras que en la parte inferior habrá una marca que solo se usará para esa función.



Imagen 17. Detectores de flanco









 [www.universae.com](http://www.universae.com)

